

Universidad Politecnica de Valencia

Grado en Informatica Industrial y Robotica

Documentacion final del proyecto

Gestor de cruce para vehiculos siguelineas con
ESP32-S3 y MQTT



Asignatura:

PR3

Grupo / autores:

Pablo Gomez, Carla Castellanos, Nuria Urrecho y
Gabriel Arnedo

Fecha:

22/03/2026

Índice

1. Introduccion	2
2. Objetivos del trabajo	2
3. Material utilizado	2
3.1. Hardware	2
3.2. Software y librerias	3
4. Descripcion general del sistema	3
4.1. Montaje fisico del circuito	3
5. Comunicacion MQTT	4
6. Funcionamiento de los robots	5
6.1. FSM principal de movimiento	5
6.2. Sub-FSM del cruce	6
7. Funcionamiento del gestor de cruce	6
7.1. Logica de control	7
7.2. Politica de acceso	7
7.3. Manejo de cancelaciones	7
8. Aportacion y ampliacion realizada	7
9. Obtencion de los tiempos de cruce	8
9.1. Criterio de medida	8
9.2. Procedimiento seguido	8
9.3. Tabla de tiempos	9
10.Resultados y funcionamiento observado	9
11.Video de demostracion	9
12.Conclusiones	10

13.Codigo fuente completo	11
13.1. Robot1.ino	11
13.2. Robot2.ino	20
13.3. placa_wifi.ino	29

1. Introduccion

El objetivo de este trabajo ha sido ampliar un sistema de vehiculos siguelineas para permitir que dos robots compartan un cruce de forma segura, ordenada y sin colisiones. Para ello se ha desarrollado un gestor de cruce basado en una placa ESP32-S3 que actua simultaneamente como punto de acceso WiFi y como broker MQTT. Cada robot se conecta a esa red, solicita permiso para entrar en el cruce y solo avanza cuando el gestor se lo concede.

La solucion implementada combina:

- Seguimiento de linea con dos sensores TCRT5000.
- Control de motores mediante Mini L298N.
- Deteccion del cruce con sensor ultrasonico HC-SR04.
- Comunicacion inalambrica mediante WiFi y MQTT.
- Gestion centralizada del acceso al cruce mediante cola FIFO.
- Maquinas de estados para desacoplar la logica de navegacion y la logica del cruce.

2. Objetivos del trabajo

Los objetivos principales del proyecto han sido los siguientes:

- Conseguir que dos robots siguelineas circulen por un trazado comun con cruce.
- Evitar que ambos entren a la vez en la misma zona de conflicto.
- Desarrollar un gestor central que arbitre el paso de manera ordenada.
- Mantener el comportamiento del siguelineas y la deteccion de obstaculos sin bloqueos.
- Mejorar la robustez del sistema frente a rebotes del sensor y perdidas del marcador del cruce.

3. Material utilizado

3.1. Hardware

- 2 placas ESP32-S3 para los robots.

- 1 placa ESP32-S3 para el gestor de cruce / broker.
- 2 sensores de linea TCRT5000 por robot.
- 1 puente H Mini L298N por robot.
- 1 sensor ultrasonico HC-SR04 por robot.
- Chasis de robot siguelineas con dos motores DC.
- Alimentacion y cableado correspondiente.

3.2. Software y librerias

- Arduino o entorno compatible con ESP32-S3.
- Libreria `WiFi.h`.
- Libreria `PicoMQTT.h`.
- Monitor serie para depuracion y toma de evidencias.

4. Descripcion general del sistema

La arquitectura final del sistema se divide en tres nodos:

- **Robot 1**, encargado de seguir la linea, detectar el cruce y obedecer al gestor.
- **Robot 2**, con el mismo comportamiento que el Robot 1 pero usando sus propios *topics* MQTT.
- **Placa gestora**, que crea la red WiFi, levanta el broker MQTT y decide que robot puede ocupar el cruce.

4.1. Montaje fisico del circuito

El circuito se construyo sobre una superficie plana utilizando cinta negra como guia de seguimiento para los robots. En la zona central se coloco una estructura cuadrada que delimita visualmente el area del cruce y obliga a que ambos vehiculos pasen por una region comun controlada por el gestor. Este montaje permite reproducir de forma clara la situacion de conflicto en la que dos robots pueden intentar acceder al mismo recurso compartido.

En la imagen puede observarse que los dos robots comparten el mismo entorno de circulacion, con trayectorias cerradas que confluyen en la zona central. Esta disposicion justifica

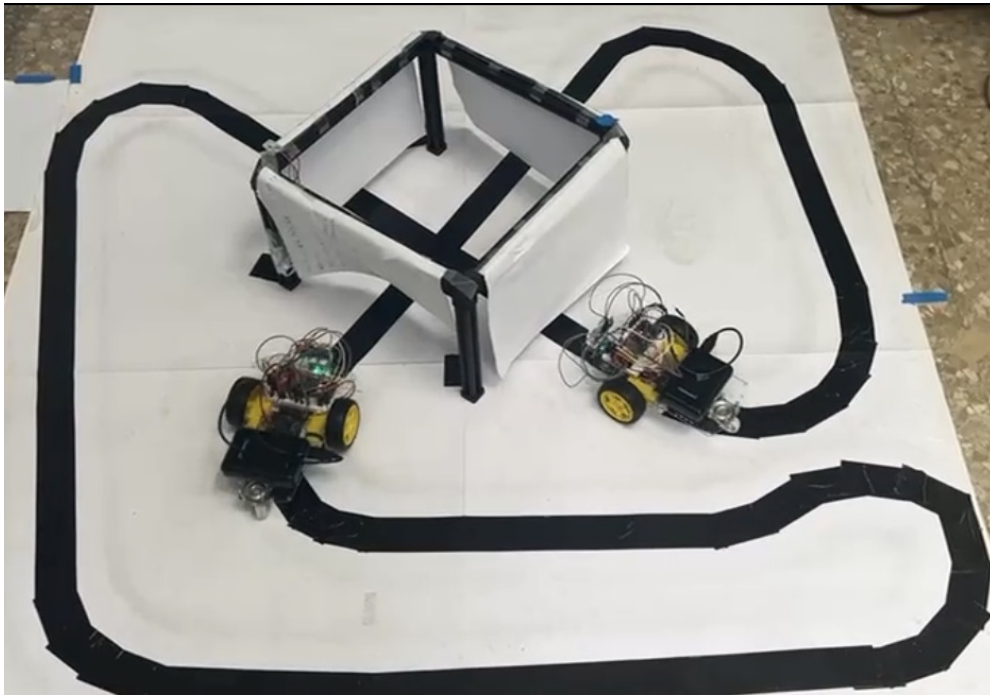


Figura 1: Montaje real del circuito y disposicion fisica del cruce.

el uso del broker MQTT y de la cola FIFO, ya que el cruce debe gestionarse como un recurso exclusivo para evitar bloqueos o colisiones.

El flujo general de funcionamiento es el siguiente:

1. La placa gestora crea una red WiFi en modo **Access Point**.
2. Los dos robots se conectan a dicha red como clientes.
3. Cuando un robot detecta el marcador de entrada del cruce, publica `{"Cruce": "Detectado"}`.
4. El gestor encola la solicitud en una FIFO.
5. Si el cruce esta libre, el gestor concede permiso al primer robot de la cola publicando `{"Permiso": "Concedido"}`.
6. El robot autorizado atraviesa el cruce.
7. Al superar el marcador de salida, el robot publica `{"Cruce": "Liberado"}`.
8. El gestor marca el cruce como libre y atiende la siguiente solicitud pendiente.

5. Comunicacion MQTT

Los *topics* principales utilizados en el proyecto son:

- `Richi/Robot1`

- Richi/Robot2
- Richi/Robot1/Estado
- Richi/Robot2/Estado

Los mensajes intercambiados son:

- {"Cruce": "Detectado"}: el robot ha localizado el marcador de entrada y solicita acceso.
- {"Cruce": "Cancelado"}: el robot pierde el marcador antes de recibir permiso y retira la solicitud.
- {"Permiso": "Concedido"}: el gestor autoriza a ese robot a cruzar.
- {"Cruce": "Liberado"}: el robot ya ha abandonado la zona del cruce.
- Publicacion del estado del robot en .../Estado: informa de la FSM principal (ADELANTE, GIRO, BUSCANDO, OBSTACULO).

6. Funcionamiento de los robots

Los archivos de los robots son Robot1.ino y Robot2.ino. Ambos programas son practicamente identicos; la diferencia principal es el *topic* MQTT utilizado por cada robot y el identificador del cliente MQTT.

6.1. FSM principal de movimiento

Cada robot implementa una maquina de estados principal con cuatro estados:

- ADELANTE: ambos sensores de linea detectan la trayectoria y el robot avanza.
- GIRO: solo uno de los sensores detecta linea y el robot corrige.
- BUSCANDO: ninguno detecta linea y el robot retrocede para recuperarla.
- OBSTACULO_DETECTADO: el robot se detiene.

La deteccion de eventos se basa en los sensores TCRT5000 para seguir la linea y en el sensor HC-SR04 para decidir si el robot debe permanecer parado. En este proyecto, el estado OBSTACULO_DETECTADO no solo representa un obstaculo fisico, sino tambien la situacion en la que el robot ha detectado el cruce pero aun no tiene permiso para avanzar.

6.2. Sub-FSM del cruce

Ademas de la FSM principal, cada robot implementa una submaquina de estados dedicada al cruce:

- FUERA_CRUCE
- ESPERANDO_PERMISO
- PERMISO_CONCEDIDO
- DENTRO_CRUCE
- SALIDA_DETECTADA

Esta separacion es una de las claves de la solucion, porque desacopla el comportamiento de navegacion del protocolo de acceso al cruce.

El comportamiento es el siguiente:

1. En **FUERA_CRUCE**, un flanco de subida del sensor ultrasonico hace que el robot solicite acceso al cruce.
2. En **ESPERANDO_PERMISO**, el robot permanece detenido hasta recibir el permiso del gestor.
3. Si el marcador desaparece antes del permiso, el robot envia **Cancelado**.
4. Al recibir **Permiso Concedido**, pasa a **PERMISO_CONCEDIDO**.
5. Tras superar el marcador de entrada, cambia a **DENTRO_CRUCE**.
6. Cuando detecta el marcador de salida, entra en **SALIDA_DETECTADA**.
7. Tras abandonar completamente el marcador de salida, envia **Liberado** y vuelve a **FUERA_CRUCE**.

7. Funcionamiento del gestor de cruce

El archivo del gestor es `placa_wifi.ino`. La placa ESP32-S3 central realiza dos funciones:

- Crear la red WiFi `ESP32S3_MQTT_RICHI`.
- Actuar como broker MQTT y arbitro del cruce.

7.1. Logica de control

El gestor mantiene:

- Una variable `cruceOcupado` para saber si el cruce esta siendo utilizado.
- El identificador `robotEnCruce` para conocer que robot tiene actualmente el permiso.
- Una cola circular FIFO para almacenar solicitudes pendientes.
- Un pequeno retardo (`RETARDO_OK_MS`) antes de conceder el permiso para estabilizar la deteccion.

7.2. Politica de acceso

La politica de acceso al cruce es estrictamente FIFO:

- Si llega una solicitud y el cruce esta libre, el robot recibira permiso en cuanto le toque en la cola.
- Si el cruce esta ocupado, la solicitud se conserva en espera.
- Cuando el robot que esta cruzando publica `Liberado`, el gestor atiende al siguiente robot pendiente.

7.3. Manejo de cancelaciones

Una mejora importante implementada en esta version es el soporte del mensaje `{"Cruce": "Cancelado"}`

Esto permite retirar de la cola a un robot que detecto el marcador de entrada pero lo perdio antes de recibir permiso. Gracias a ello:

- No quedan robots fantasma bloqueando la cola.
- La FIFO refleja mejor el estado real del sistema.
- El gestor sigue siendo coherente incluso con detecciones fugaces o rebotes.

8. Aportacion y ampliacion realizada

La parte mas relevante de nuestra aportacion no ha sido solo conectar dos robots por WiFi, sino integrar una solucion completa de coordinacion distribuida con control centralizado del cruce. En concreto, las ampliaciones desarrolladas han sido:

- Implementacion de una cola FIFO para arbitrar el orden de paso.
- Uso de dos marcadores fisicos en el cruce: uno de entrada y otro de salida.
- Cancelacion de solicitudes si el robot pierde el marcador antes de obtener permiso.
- Deteccion no bloqueante del ultrasonido mediante interrupciones (ISR).
- Debounce temporal de flancos para filtrar rebotes del sensor.
- Integracion de una FSM principal de movimiento con una sub-FSM especifica del cruce.

Estas ampliaciones hacen que el sistema sea mas robusto, escalable y cercano a un escenario real de coordinacion de trafico entre agentes autonomos.

9. Obtencion de los tiempos de cruce

9.1. Criterio de medida

En las pruebas finales, el tiempo de cruce se ha medido manualmente a partir del video de demostracion. El criterio utilizado ha sido el siguiente:

- **Inicio de medida:** instante en el que el robot detecta el papel de entrada del cruce.
- **Fin de medida:** instante en el que el robot abandona completamente el cruce.

De esta forma, la medida refleja el tiempo total que transcurre desde la deteccion de la zona de cruce hasta la salida completa del robot, incluyendo en el caso del Robot 2 el tiempo de espera antes de recibir paso.

9.2. Procedimiento seguido

El procedimiento seguido para obtener los tiempos ha sido:

1. Reproducir el video de demostracion.
2. Identificar el instante en el que cada robot detecta el papel de entrada.
3. Identificar el instante en el que cada robot sale completamente del cruce.
4. Calcular la diferencia entre ambos tiempos.
5. Comparar el resultado del robot que cruza primero con el del robot que debe esperar.

9.3. Tabla de tiempos

En las pruebas mostradas en el video, el tiempo se ha medido en segundos desde que el robot detecta el papel de entrada del cruce hasta que abandona completamente el cruce. En este caso, el Robot 1 es el que llega primero al cruce y el Robot 2 es el que espera su turno.

Robot	Inicio (s)	Fin (s)	Tiempo (s)
Robot 1	3,39	6,09	2,70
Robot 2	3,59	7,26	3,67

Se observa que el Robot 2 presenta un tiempo mayor porque debe esperar a que el Robot 1 libere el cruce antes de poder avanzar.

10. Resultados y funcionamiento observado

Durante las pruebas realizadas, el sistema presenta el comportamiento esperado:

- Los robots no entran simultaneamente al cruce.
- El orden de paso se respeta segun la cola FIFO.
- Un robot puede quedar esperando mientras otro esta ocupando el cruce.
- Si un robot pierde el marcador antes de recibir permiso, la solicitud se cancela correctamente.
- El sistema conserva la navegacion siguelineas fuera del cruce.

11. Video de demostracion

Enlace al video: <https://youtu.be/Z9xnEcBih1I?is=cBlazxKgYBjD3NJV>

En el video se observa:

- Ambos robots funcionando sobre el circuito.
- Llegada de dos robots al cruce.
- Espera del segundo robot mientras el primero cruza.
- Liberacion del cruce y paso del siguiente robot.

12. Conclusiones

El proyecto ha permitido integrar control embebido, comunicaciones inalambricas y coordinacion entre agentes autonomos en un unico sistema funcional. La solucion propuesta evita colisiones en el cruce, mantiene un comportamiento ordenado mediante cola FIFO y mejora la robustez del sistema con cancelacion de solicitudes, deteccion no bloqueante y filtrado de rebotes.

La ampliacion realizada aporta un nivel de complejidad superior al de un siguelineas convencional, ya que introduce comunicacion en red, sincronizacion entre nodos y arbitraje de recursos compartidos. El resultado final cumple con el objetivo del trabajo y demuestra el funcionamiento coordinado del gestor de cruce y de los dos vehiculos.

13. Código fuente completo

13.1. Robot1.ino

```
/*
  ROBOT 1 - SIGUELÍNEAS ESP32-S3
  2 sensores TCRT5000 + Mini L298N + HC-SR04
  + MQTT Cruce Inteligente
  VERSIÓN SIN TIEMPOS + ULTRASONIDO NO BLOQUEANTE (ISR) + DEBOUNCE FLANCOS
*/

#include <Arduino.h>
#include <WiFi.h>
#include <PicoMQTT.h>

/* ----- PINES MOTORES ----- */
#define IN1 4
#define IN2 5
#define IN3 6
#define IN4 7

/* ----- LEDS ----- */
#define LED_VERDE 12
#define LED_ROJO 13

/* ----- WIFI ----- */
const char* WIFI_SSID = "ESP32S3_MQTT_RICHI";
const char* WIFI_PASS = "12345678";

/* ----- MQTT ----- */
const char* BROKER_IP = "192.168.4.1";
const int BROKER_PORT = 1883;
const char* TOPIC_ROBOT = "Richi/Robot1";
const char* TOPIC_ESTADO = "Richi/Robot1/Estado";

PicoMQTT::Client mqtt(BROKER_IP, BROKER_PORT, "cliente_s3_robot1");

/* ----- PWM ----- */
const int PWM_FREQ = 5000;
const int PWM_RES = 8;

/* ----- VELOCIDADES ----- */
int VEL_BASE = 205;
int VEL_BOOST = 220;
```

```
int CORRECCION    = 20;
int VEL_BUSCANDO  = 200;

/* ----- SENSORES DE LÍNEA ----- */
#define SENSOR_IZQ 15
#define SENSOR_DER 16

/* ----- ULTRASONIDO NO BLOQUEANTE ----- */
#define TRIG_PIN 17
#define ECHO_PIN 18

const float UMBRAL_DETECTA_CM      = 4.0f;
const float UMBRAL_LIBRE_CM        = 6.0f;
const unsigned long INTERVALO_TRIGGER_MS = 25;

/* Tiempo mínimo entre dos flancos válidos consecutivos.
   A 30 cm/s el robot tarda ~130 ms en atravesar un papel de 4 cm.
   Con 80 ms filtramos rebotes sin perder papeles reales. */
const unsigned long DEBOUNCE_FLANCO_MS = 80;

volatile unsigned long echoStart    = 0;
volatile unsigned long echoDuracion = 0;
volatile bool          echoListo    = false;

float          distanciaActual = 100.0f;
unsigned long tUltimoTrigger    = 0;

/* Estado del sensor con debounce */
bool          sensorActivo      = false;
bool          sensorActivoPrev  = false;
unsigned long tUltimoFlanco      = 0;  // millis() del último flanco aceptado

/* ISR */
void IRAM_ATTR echoISR() {
    if (digitalRead(ECHO_PIN) == HIGH) {
        echoStart = micros();
    } else {
        unsigned long dur = micros() - echoStart;
        if (dur < 30000UL) {
            echoDuracion = dur;
            echoListo     = true;
        }
    }
}
```

```
/* Trigger periódico + lectura de eco */
void actualizarUltrasonido() {
    unsigned long ahora = millis();
    if (ahora - tUltimoTrigger >= INTERVALO_TRIGGER_MS) {
        tUltimoTrigger = ahora;
        digitalWrite(TRIG_PIN, LOW);
        delayMicroseconds(2);
        digitalWrite(TRIG_PIN, HIGH);
        delayMicroseconds(10);
        digitalWrite(TRIG_PIN, LOW);
    }

    if (echoListo) {
        noInterrupts();
        unsigned long dur = echoDuracion;
        echoListo = false;
        interrupts();
        distanciaActual = dur * 0.034f / 2.0f;
    }
}

/* =====
   SUB-FSM DEL CRUCE -- sin millis() en la lógica de cruce
   ===== */
enum EstadoCruce : uint8_t {
    FUERA_CRUCE,
    ESPERANDO_PERMISO,
    PERMISO_CONCEDIDO,
    DENTRO_CRUCE,
    SALIDA_DETECTADA
};

EstadoCruce estadoCruce = FUERA_CRUCE;

/* ----- FSM PRINCIPAL ----- */
enum Estado : uint8_t {
    ADELANTE,
    GIRO,
    BUSCANDO,
    OBSTACULO_DETECTADO
};

enum Evento : uint32_t {
    EV_NONE = 0,
    EV_LINEA_IZQ = 1u << 0,
```

```
EV_LINEA_DER = 1u << 1,
EV_OBSTACULO = 1u << 2,
EV_LIBRE      = 1u << 3
};

Estado estadoActual  = ADELANTE;
Estado estadoAnterior = ADELANTE;
int8_t dirGiro        = +1;

/* ----- PROTOTIPOS ----- */
const char* nombreEstado(Estado s);
const char* nombreEstadoCruce(EstadoCruce s);
void moverMotores(int velIzq, int velDer);
uint32_t detectarEventos(int &izq, int &der);
Estado determinarSiguienteEstado(Estado s, uint32_t ev);
void ejecutarEstado(Estado s);

/* ----- LEDS ----- */
void ledsNormal()      { digitalWrite(LED_VERDE, HIGH); digitalWrite(LED_ROJO,
    LOW); }
void ledsSolicitando() { digitalWrite(LED_VERDE, LOW);  digitalWrite(LED_ROJO,
    HIGH); }
void ledsConectando() {
    static unsigned long t = 0; static bool tog = false;
    if (millis() - t > 300) { t = millis(); tog = !tog;
        digitalWrite(LED_VERDE, tog); digitalWrite(LED_ROJO, !tog); }
}

/* ----- NOMBRES ----- */
const char* nombreEstado(Estado s) {
    switch (s) {
        case ADELANTE:      return "ADELANTE";
        case GIRO:          return "GIRO";
        case BUSCANDO:      return "BUSCANDO";
        case OBSTACULO_DETECTADO: return "OBSTACULO";
        default:            return "?";
    }
}

const char* nombreEstadoCruce(EstadoCruce s) {
    switch (s) {
        case FUERA_CRUCE:    return "FUERA";
        case ESPERANDO_PERMISO: return "ESPERANDO";
        case PERMISO_CONCEDIDO: return "PERMISO_OK";
        case DENTRO_CRUCE:   return "DENTRO";
        case SALIDA_DETECTADA: return "SALIDA";
    }
}
```



```
    default:                return "?";
  }
}

/* ----- MOTORES ----- */
void moverMotores(int velIzq, int velDer) {
  velIzq = constrain(velIzq, -255, 255);
  velDer = constrain(velDer, -255, 255);
  ledcWrite(IN1, velIzq >= 0 ? velIzq : 0);
  ledcWrite(IN2, velIzq < 0 ? -velIzq : 0);
  ledcWrite(IN3, velDer >= 0 ? velDer : 0);
  ledcWrite(IN4, velDer < 0 ? -velDer : 0);
}

/* =====
   SUB-FSM DEL CRUCE
   ===== */
void actualizarEstadoCruce(bool flancoSubida, bool flancoBajada) {
  switch (estadoCruce) {

    case FUERA_CRUCE:
      if (flancoSubida) {
        estadoCruce = ESPERANDO_PERMISO;
        mqtt.publish(TOPIC_ROBOT, "{\"Cruce\":\"Detectado\"}");
        ledsSolicitando();
        Serial.println("[MQTT] Marcador ENTRADA -> solicitud enviada");
      }
      break;

    case ESPERANDO_PERMISO:
      if (flancoBajada) {
        estadoCruce = FUERA_CRUCE;
        mqtt.publish(TOPIC_ROBOT, "{\"Cruce\":\"Cancelado\"}");
        Serial.println("[WARN] Marcador perdido antes del permiso -> cancelado");
      }
      break;

    case PERMISO_CONCEDIDO:
      if (flancoBajada) {
        estadoCruce = DENTRO_CRUCE;
        Serial.println("[IN] Marcador ENTRADA superado -> DENTRO_CRUCE");
      }
      break;
  }
}
```

```
case DENTRO_CRUCE:
    if (flancoSubida) {
        estadoCruce = SALIDA_DETECTADA;
        Serial.println("[OUT] Marcador SALIDA detectado");
    }
    break;

case SALIDA_DETECTADA:
    if (flancoBajada) {
        mqtt.publish(TOPIC_ROBOT, "{\"Cruce\":\"Liberado\"}");
        estadoCruce = FUERA_CRUCE;
        ledsNormal();
        Serial.println("[GO] Marcador SALIDA superado -> cruce LIBERADO");
    }
    break;
}
}

/* ----- DETECTAR EVENTOS ----- */
uint32_t detectarEventos(int &izq, int &der) {
    uint32_t ev = EV_NONE;

    izq = digitalRead(SENSOR_IZQ);
    der = digitalRead(SENSOR_DER);
    if (izq == HIGH) ev |= EV_LINEA_IZQ;
    if (der == HIGH) ev |= EV_LINEA_DER;

    actualizarUltrasonido();

    /* --- Debounce de flancos del sensor --- */
    bool nuevoEstado = sensorActivo;
    if (!sensorActivo && distanciaActual <= UMBRAL_DETECTA_CM) nuevoEstado = true
    ;
    if ( sensorActivo && distanciaActual >  UMBRAL_LIBRE_CM)    nuevoEstado =
        false;

    bool flancoSubida = false;
    bool flancoBajada = false;

    if (nuevoEstado != sensorActivo) {
        /* Hay un cambio -- ¿ha pasado suficiente tiempo desde el último flanco? */
        unsigned long ahora = millis();
        if (ahora - tUltimoFlanco >= DEBOUNCE_FLANCO_MS) {
            tUltimoFlanco = ahora;
            sensorActivoPrev = sensorActivo;
        }
    }
}
```

```
    sensorActivo      = nuevoEstado;

    flancoSubida = ( sensorActivo && !sensorActivoPrev);
    flancoBajada = (!sensorActivo &&  sensorActivoPrev);

    Serial.printf("[EDGE] Flanco %s | dist=%.1f\n",
                  flancoSubida ? "SUBIDA" : "BAJADA", distanciaActual);
} else {
    Serial.printf("[DB] Flanco ignorado (debounce) | dist=%.1f\n",
                  distanciaActual);
}
}

actualizarEstadoCruce(flancoSubida, flancoBajada);

if (estadoCruce == ESPERANDO_PERMISO) ev |= EV_OBSTACULO;
else                                ev |= EV_LIBRE;

static unsigned long tLog = 0;
if (millis() - tLog > 100) {
    tLog = millis();
    Serial.printf("IZQ=%d DER=%d dist=%.1f FSM=%s CRUCE=%s\n",
                  izq, der, distanciaActual,
                  nombreEstado(estadoActual),
                  nombreEstadoCruce(estadoCruce));
}

return ev;
}

/* ----- FSM PRINCIPAL ----- */
Estado determinarSiguienteEstado(Estado s, uint32_t ev) {
    if (ev & EV_OBSTACULO) return OBSTACULO_DETECTADO;
    if (s == OBSTACULO_DETECTADO && !(ev & EV_LIBRE)) return OBSTACULO_DETECTADO;

    bool L = ev & EV_LINEA_IZQ;
    bool R = ev & EV_LINEA_DER;
    if (L && R) return ADELANTE;
    if (L)     { dirGiro = -1; return GIRO; }
    if (R)     { dirGiro = +1; return GIRO; }
    return BUSCANDO;
}

/* ----- EJECUTAR ESTADO ----- */
void ejecutarEstado(Estado s) {
```

```
switch (s) {
  case ADELANTE:
    moverMotores(
      (estadoCruce == PERMISO_CONCEDIDO || estadoCruce == DENTRO_CRUCE) ?
      VEL_BOOST : VEL_BASE,
      (estadoCruce == PERMISO_CONCEDIDO || estadoCruce == DENTRO_CRUCE) ?
      VEL_BOOST : VEL_BASE);
    ledsNormal();
    break;
  case GIRO:
    ledsNormal();
    if (dirGiro < 0) moverMotores(0, VEL_BASE + CORRECCION);
    else          moverMotores(VEL_BASE + CORRECCION, 0);
    break;
  case BUSCANDO:
    ledsNormal();
    moverMotores(-VEL_BUSCANDO, -VEL_BUSCANDO);
    break;
  case OBSTACULO_DETECTADO:
    moverMotores(0, 0);
    ledsSolicitando();
    break;
}
}

/* ----- SETUP ----- */
void setup() {
  Serial.begin(115200);

  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_ROJO,  OUTPUT);

  ledcAttach(IN1, PWM_FREQ, PWM_RES);
  ledcAttach(IN2, PWM_FREQ, PWM_RES);
  ledcAttach(IN3, PWM_FREQ, PWM_RES);
  ledcAttach(IN4, PWM_FREQ, PWM_RES);

  pinMode(SENSOR_IZQ, INPUT);
  pinMode(SENSOR_DER, INPUT);
  pinMode(TRIG_PIN, OUTPUT);
  digitalWrite(TRIG_PIN, LOW);
  pinMode(ECHO_PIN, INPUT);

  attachInterrupt(digitalPinToInterrupt(ECHO_PIN), echoISR, CHANGE);
```

```
estadoActual      = ADELANTE;
estadoCruce       = FUERA_CRUCE;
sensorActivo      = false;
sensorActivoPrev  = false;
tUltimoFlanco     = 0;

Serial.println("Robot 1 listo -- ISR + debounce flancos");

WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASS);
while (WiFi.status() != WL_CONNECTED) ledsConectando();
ledsNormal();

mqtt.begin();

mqtt.subscribe(TOPIC_ROBOT, [](const char* topic, const void* payload, size_t
    size) {
    String msg;
    for (size_t i = 0; i < size; i++) msg += (char)((uint8_t*)payload)[i];

    if (msg.indexOf("Permiso") != -1 && msg.indexOf("Concedido") != -1) {
        if (estadoCruce == ESPERANDO_PERMISO) {
            estadoCruce = PERMISO_CONCEDIDO;
            ledsNormal();
            Serial.println("[OK] Permiso concedido -> arrancando");
        }
    }
});
}

/* ----- LOOP -- sin delay() ----- */
void loop() {
    mqtt.loop();

    int izq, der;
    uint32_t ev = detectarEventos(izq, der);
    Estado sig = determinarSiguienteEstado(estadoActual, ev);

    if (sig != estadoActual) {
        estadoAnterior = estadoActual;
        estadoActual = sig;
        mqtt.publish(TOPIC_ESTADO, nombreEstado(estadoActual));
        Serial.printf("Cambio FSM: %s -> %s\n",
            nombreEstado(estadoAnterior),
            nombreEstado(estadoActual));
    }
}
```

```
}

    ejecutarEstado(estadoActual);
}
```

13.2. Robot2.ino

```
/*
  ROBOT 2 - SIGUELÍNEAS ESP32-S3
  2 sensores TCRT5000 + Mini L298N + HC-SR04
  + MQTT Cruce Inteligente
  VERSIÓN SIN TIEMPOS + ULTRASONIDO NO BLOQUEANTE (ISR) + DEBOUNCE FLANCOS
*/

#include <Arduino.h>
#include <WiFi.h>
#include <PicoMQTT.h>

/* ----- PINES MOTORES ----- */
#define IN1 4
#define IN2 5
#define IN3 6
#define IN4 7

/* ----- LEDS ----- */
#define LED_VERDE 12
#define LED_ROJO 13

/* ----- WIFI ----- */
const char* WIFI_SSID = "ESP32S3_MQTT_RICHI";
const char* WIFI_PASS = "12345678";

/* ----- MQTT ----- */
const char* BROKER_IP = "192.168.4.1";
const int BROKER_PORT = 1883;
const char* TOPIC_ROBOT = "Richi/Robot2";
const char* TOPIC_ESTADO = "Richi/Robot2/Estado";

PicoMQTT::Client mqtt(BROKER_IP, BROKER_PORT, "cliente_s3_robot2");

/* ----- PWM ----- */
const int PWM_FREQ = 5000;
const int PWM_RES = 8;
```

```
/* ----- VELOCIDADES ----- */
int VEL_BASE      = 205;
int VEL_BOOST     = 220;
int CORRECCION    = 20;
int VEL_BUSCANDO  = 200;

/* ----- SENSORES DE LÍNEA ----- */
#define SENSOR_IZQ 15
#define SENSOR_DER 16

/* ----- ULTRASONIDO NO BLOQUEANTE ----- */
#define TRIG_PIN 17
#define ECHO_PIN 18

const float UMBRAL_DETECTA_CM      = 4.0f;
const float UMBRAL_LIBRE_CM        = 6.0f;
const unsigned long INTERVALO_TRIGGER_MS = 25;
const unsigned long DEBOUNCE_FLANCO_MS  = 80;

volatile unsigned long echoStart      = 0;
volatile unsigned long echoDuracion = 0;
volatile bool          echoListo      = false;

float          distanciaActual = 100.0f;
unsigned long tUltimoTrigger    = 0;

bool          sensorActivo      = false;
bool          sensorActivoPrev = false;
unsigned long tUltimoFlanco     = 0;

void IRAM_ATTR echoISR() {
    if (digitalRead(ECHO_PIN) == HIGH) {
        echoStart = micros();
    } else {
        unsigned long dur = micros() - echoStart;
        if (dur < 30000UL) {
            echoDuracion = dur;
            echoListo     = true;
        }
    }
}

void actualizarUltrasonido() {
    unsigned long ahora = millis();
    if (ahora - tUltimoTrigger >= INTERVALO_TRIGGER_MS) {
```

```
tUltimoTrigger = ahora;
digitalWrite(TRIG_PIN, LOW);
delayMicroseconds(2);
digitalWrite(TRIG_PIN, HIGH);
delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);
}

if (echoListo) {
    noInterrupts();
    unsigned long dur = echoDuracion;
    echoListo = false;
    interrupts();
    distanciaActual = dur * 0.034f / 2.0f;
}
}

/* =====
   SUB-FSM DEL CRUCE
   ===== */
enum EstadoCruce : uint8_t {
    FUERA_CRUCE,
    ESPERANDO_PERMISO,
    PERMISO_CONCEDIDO,
    DENTRO_CRUCE,
    SALIDA_DETECTADA
};

EstadoCruce estadoCruce = FUERA_CRUCE;

/* ----- FSM PRINCIPAL ----- */
enum Estado : uint8_t {
    ADELANTE,
    GIRO,
    BUSCANDO,
    OBSTACULO_DETECTADO
};

enum Evento : uint32_t {
    EV_NONE      = 0,
    EV_LINEA_IZQ = 1u << 0,
    EV_LINEA_DER = 1u << 1,
    EV_OBSTACULO = 1u << 2,
    EV_LIBRE     = 1u << 3
};
```



```
Estado estadoActual    = ADELANTE;
Estado estadoAnterior = ADELANTE;
int8_t dirGiro          = +1;

/* ----- PROTOTIPOS ----- */
const char* nombreEstado(Estado s);
const char* nombreEstadoCruce(EstadoCruce s);
void moverMotores(int velIzq, int velDer);
uint32_t detectarEventos(int &izq, int &der);
Estado determinarSiguienteEstado(Estado s, uint32_t ev);
void ejecutarEstado(Estado s);

/* ----- LEDS ----- */
void ledsNormal()      { digitalWrite(LED_VERDE, HIGH); digitalWrite(LED_ROJO,
    LOW); }
void ledsSolicitando() { digitalWrite(LED_VERDE, LOW);  digitalWrite(LED_ROJO,
    HIGH); }
void ledsConectando()  {
    static unsigned long t = 0; static bool tog = false;
    if (millis() - t > 300) { t = millis(); tog = !tog;
        digitalWrite(LED_VERDE, tog); digitalWrite(LED_ROJO, !tog); }
}

/* ----- NOMBRES ----- */
const char* nombreEstado(Estado s) {
    switch (s) {
        case ADELANTE:      return "ADELANTE";
        case GIRO:          return "GIRO";
        case BUSCANDO:      return "BUSCANDO";
        case OBSTACULO_DETECTADO: return "OBSTACULO";
        default:            return "?";
    }
}

const char* nombreEstadoCruce(EstadoCruce s) {
    switch (s) {
        case FUERA_CRUCE:    return "FUERA";
        case ESPERANDO_PERMISO: return "ESPERANDO";
        case PERMISO_CONCEDIDO: return "PERMISO_OK";
        case DENTRO_CRUCE:   return "DENTRO";
        case SALIDA_DETECTADA: return "SALIDA";
        default:            return "?";
    }
}
```

```
/* ----- MOTORES ----- */
void moverMotores(int velIzq, int velDer) {
    velIzq = constrain(velIzq, -255, 255);
    velDer = constrain(velDer, -255, 255);
    ledcWrite(IN1, velIzq >= 0 ? velIzq : 0);
    ledcWrite(IN2, velIzq < 0 ? -velIzq : 0);
    ledcWrite(IN3, velDer >= 0 ? velDer : 0);
    ledcWrite(IN4, velDer < 0 ? -velDer : 0);
}

/* =====
   SUB-FSM DEL CRUCE
   ===== */
void actualizarEstadoCruce(bool flancoSubida, bool flancoBajada) {
    switch (estadoCruce) {

        case FUERA_CRUCE:
            if (flancoSubida) {
                estadoCruce = ESPERANDO_PERMISO;
                mqtt.publish(TOPIC_ROBOT, "{\"Cruce\":\"Detectado\"}");
                ledsSolicitando();
                Serial.println("[MQTT] Marcador ENTRADA -> solicitud enviada");
            }
            break;

        case ESPERANDO_PERMISO:
            if (flancoBajada) {
                estadoCruce = FUERA_CRUCE;
                mqtt.publish(TOPIC_ROBOT, "{\"Cruce\":\"Cancelado\"}");
                Serial.println("[WARN] Marcador perdido antes del permiso -> cancelado");
            }
            break;

        case PERMISO_CONCEDIDO:
            if (flancoBajada) {
                estadoCruce = DENTRO_CRUCE;
                Serial.println("[IN] Marcador ENTRADA superado -> DENTRO_CRUCE");
            }
            break;

        case DENTRO_CRUCE:
            if (flancoSubida) {
                estadoCruce = SALIDA_DETECTADA;
                Serial.println("[OUT] Marcador SALIDA detectado");
            }
    }
}
```

```
    }
    break;

    case SALIDA_DETECTADA:
        if (flancoBajada) {
            mqtt.publish(TOPIC_ROBOT, "{\"Cruce\":\"Liberado\"}");
            estadoCruce = FUERA_CRUCE;
            ledsNormal();
            Serial.println("[GO] Marcador SALIDA superado -> cruce LIBERADO");
        }
        break;
    }
}

/* ----- DETECTAR EVENTOS ----- */
uint32_t detectarEventos(int &izq, int &der) {
    uint32_t ev = EV_NONE;

    izq = digitalRead(SENSOR_IZQ);
    der = digitalRead(SENSOR_DER);
    if (izq == HIGH) ev |= EV_LINEA_IZQ;
    if (der == HIGH) ev |= EV_LINEA_DER;

    actualizarUltrasonido();

    /* --- Debounce de flancos --- */
    bool nuevoEstado = sensorActivo;
    if (!sensorActivo && distanciaActual <= UMBRAL_DETECTA_CM) nuevoEstado = true
    ;
    if ( sensorActivo && distanciaActual >  UMBRAL_LIBRE_CM)    nuevoEstado =
        false;

    bool flancoSubida = false;
    bool flancoBajada = false;

    if (nuevoEstado != sensorActivo) {
        unsigned long ahora = millis();
        if (ahora - tUltimoFlanco >= DEBOUNCE_FLANCO_MS) {
            tUltimoFlanco      = ahora;
            sensorActivoPrev = sensorActivo;
            sensorActivo      = nuevoEstado;

            flancoSubida = ( sensorActivo && !sensorActivoPrev);
            flancoBajada = (!sensorActivo &&  sensorActivoPrev);
        }
    }
}
```

```
        Serial.printf("[EDGE] Flanco %s | dist=%.1f\n",
                      flancoSubida ? "SUBIDA" : "BAJADA", distanciaActual);
    } else {
        Serial.printf("[DB] Flanco ignorado (debounce) | dist=%.1f\n",
                      distanciaActual);
    }
}

actualizarEstadoCruce(flancoSubida, flancoBajada);

if (estadoCruce == ESPERANDO_PERMISO) ev |= EV_OBSTACULO;
else ev |= EV_LIBRE;

static unsigned long tLog = 0;
if (millis() - tLog > 100) {
    tLog = millis();
    Serial.printf("IZQ=%d DER=%d dist=%.1f FSM=%s CRUCE=%s\n",
                  izq, der, distanciaActual,
                  nombreEstado(estadoActual),
                  nombreEstadoCruce(estadoCruce));
}

return ev;
}

/* ----- FSM PRINCIPAL ----- */
Estado determinarSiguienteEstado(Estado s, uint32_t ev) {
    if (ev & EV_OBSTACULO) return OBSTACULO_DETECTADO;
    if (s == OBSTACULO_DETECTADO && !(ev & EV_LIBRE)) return OBSTACULO_DETECTADO;

    bool L = ev & EV_LINEA_IZQ;
    bool R = ev & EV_LINEA_DER;
    if (L && R) return ADELANTE;
    if (L) { dirGiro = -1; return GIRO; }
    if (R) { dirGiro = +1; return GIRO; }
    return BUSCANDO;
}

/* ----- EJECUTAR ESTADO ----- */
void ejecutarEstado(Estado s) {
    switch (s) {
        case ADELANTE:
            moverMotores(
                (estadoCruce == PERMISO_CONCEDIDO || estadoCruce == DENTRO_CRUCE) ?
                VEL_BOOST : VEL_BASE,
```

```
        (estadoCruce == PERMISO_CONCEDIDO || estadoCruce == DENTRO_CRUCE) ?
VEL_BOOST : VEL_BASE);
    ledsNormal();
    break;
case GIRO:
    ledsNormal();
    if (dirGiro < 0) moverMotores(0, VEL_BASE + CORRECCION);
    else
        moverMotores(VEL_BASE + CORRECCION, 0);
    break;
case BUSCANDO:
    ledsNormal();
    moverMotores(-VEL_BUSCANDO, -VEL_BUSCANDO);
    break;
case OBSTACULO_DETECTADO:
    moverMotores(0, 0);
    ledsSolicitando();
    break;
}
}

/* ----- SETUP ----- */
void setup() {
    Serial.begin(115200);

    pinMode(LED_VERDE, OUTPUT);
    pinMode(LED_ROJO, OUTPUT);

    ledcAttach(IN1, PWM_FREQ, PWM_RES);
    ledcAttach(IN2, PWM_FREQ, PWM_RES);
    ledcAttach(IN3, PWM_FREQ, PWM_RES);
    ledcAttach(IN4, PWM_FREQ, PWM_RES);

    pinMode(SENSOR_IZQ, INPUT);
    pinMode(SENSOR_DER, INPUT);
    pinMode(TRIG_PIN, OUTPUT);
    digitalWrite(TRIG_PIN, LOW);
    pinMode(ECHO_PIN, INPUT);

    attachInterrupt(digitalPinToInterrupt(ECHO_PIN), echoISR, CHANGE);

    estadoActual      = ADELANTE;
    estadoCruce       = FUERA_CRUCE;
    sensorActivo       = false;
    sensorActivoPrev  = false;
    tUltimoFlanco      = 0;
```

```
Serial.println("Robot 2 listo -- ISR + debounce flancos");

WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASS);
while (WiFi.status() != WL_CONNECTED) ledsConectando();
ledsNormal();

mqtt.begin();

mqtt.subscribe(TOPIC_ROBOT, [](const char* topic, const void* payload, size_t
    size) {
    String msg;
    for (size_t i = 0; i < size; i++) msg += (char)((uint8_t*)payload)[i];

    if (msg.indexOf("Permiso") != -1 && msg.indexOf("Concedido") != -1) {
        if (estadoCruce == ESPERANDO_PERMISO) {
            estadoCruce = PERMISO_CONCEDIDO;
            ledsNormal();
            Serial.println("[OK] Permiso concedido -> arrancando");
        }
    }
});
}

/* ----- LOOP -- sin delay() ----- */
void loop() {
    mqtt.loop();

    int izq, der;
    uint32_t ev = detectarEventos(izq, der);
    Estado sig = determinarSiguienteEstado(estadoActual, ev);

    if (sig != estadoActual) {
        estadoAnterior = estadoActual;
        estadoActual = sig;
        mqtt.publish(TOPIC_ESTADO, nombreEstado(estadoActual));
        Serial.printf("Cambio FSM: %s -> %s\n",
            nombreEstado(estadoAnterior),
            nombreEstado(estadoActual));
    }

    ejecutarEstado(estadoActual);
}
```

13.3. placa_wifi.ino

```
/*
  BROKER / CONTROLADOR DE CRUCE
  ESP32-S3 en modo Access Point
  VERSION SIN TIEMPOS - compatible con dos marcadores fisicos

  Cambio respecto a la version anterior:
  - Acepta mensaje {"Cruce":"Cancelado"} para retirar al robot de la cola
    si perdio el marcador antes de recibir permiso.
*/

#include <WiFi.h>
#include <PicoMQTT.h>

/* ----- WIFI AP ----- */
const char* AP_SSID = "ESP32S3_MQTT_RICHI";
const char* AP_PASS = "12345678";

/* ----- MQTT Broker ----- */
PicoMQTT::Server mqtt;

/* ----- Topics ----- */
const char* TOPIC_R1 = "Richi/Robot1";
const char* TOPIC_R2 = "Richi/Robot2";

/* ----- Control Cruce ----- */
bool cruceOcupado = false;
char robotEnCruce[32] = {0};

/* ----- Retardo minimo antes de dar OK ----- */
const unsigned long RETARDO_OK_MS = 100;

bool permisoPendiente = false;
unsigned long tSolicitud = 0;
char topicPendiente[32] = {0};

/* ===== FIFO (cola circular) ===== */
static const uint8_t FIFO_MAX = 10;
char    fifo[FIFO_MAX][32];
uint8_t head  = 0;
uint8_t tail  = 0;
uint8_t count = 0;

bool fifoContains(const char* topic) {
```

```
for (uint8_t i = 0; i < count; i++) {
    uint8_t idx = (head + i) % FIFO_MAX;
    if (strcmp(fifo[idx], topic) == 0) return true;
}
return false;
}

bool fifoPush(const char* topic) {
    if (count >= FIFO_MAX) return false;
    if (fifoContains(topic)) return true;    // ya estaba, no duplicar

    strncpy(fifo[tail], topic, sizeof(fifo[tail]) - 1);
    fifo[tail][sizeof(fifo[tail]) - 1] = '\0';
    tail = (tail + 1) % FIFO_MAX;
    count++;
    return true;
}

bool fifoPop(char* out, size_t outSize) {
    if (count == 0) return false;
    strncpy(out, fifo[head], outSize - 1);
    out[outSize - 1] = '\0';
    head = (head + 1) % FIFO_MAX;
    count--;
    return true;
}

/* Elimina un topic concreto de la cola (cancelacion) */
bool fifoRemove(const char* topic) {
    bool found = false;
    uint8_t newCount = 0;
    char tmp[FIFO_MAX][32];

    for (uint8_t i = 0; i < count; i++) {
        uint8_t idx = (head + i) % FIFO_MAX;
        if (!found && strcmp(fifo[idx], topic) == 0) {
            found = true;    // descarta esta entrada
        } else {
            strncpy(tmp[newCount], fifo[idx], sizeof(tmp[newCount]) - 1);
            tmp[newCount][sizeof(tmp[newCount]) - 1] = '\0';
            newCount++;
        }
    }
}

if (found) {
```



```

    // Reconstruir FIFO lineal desde head=0
    head = 0;
    tail = newCount;
    count = newCount;
    for (uint8_t i = 0; i < newCount; i++) {
        strncpy(fifo[i], tmp[i], sizeof(fifo[i]) - 1);
        fifo[i][sizeof(fifo[i]) - 1] = '\0';
    }
}
return found;
}

void fifoPrint() {
    Serial.print("[FIFO] FIFO (");
    Serial.print(count);
    Serial.print("): ");
    for (uint8_t i = 0; i < count; i++) {
        uint8_t idx = (head + i) % FIFO_MAX;
        Serial.print(fifo[idx]);
        if (i + 1 < count) Serial.print(" -> ");
    }
    Serial.println();
}

/* ===== */

void concederPermiso(const char* topicRobot) {
    mqtt.publish(topicRobot, "{\"Permiso\":\"Concedido\"}");

    cruceOcupado = true;
    strncpy(robotEnCruce, topicRobot, sizeof(robotEnCruce) - 1);
    robotEnCruce[sizeof(robotEnCruce) - 1] = '\0';

    Serial.print("[OK] Permiso concedido a: ");
    Serial.println(topicRobot);
    Serial.print("[CAR] En cruce ahora: ");
    Serial.println(robotEnCruce);
}

void manejarMensaje(const char* topic, const void* payload, size_t payload_size) {
    String msg;
    msg.reserve(payload_size);
    for (size_t i = 0; i < payload_size; i++)
        msg += (char)((const uint8_t*)payload)[i];
}

```

```
Serial.print("[MSG] De ");
Serial.print(topic);
Serial.print(": ");
Serial.println(msg);

// 1) Solicitud de entrada
if (msg.indexOf("Cruce") != -1 && msg.indexOf("Detectado") != -1) {
  if (fifoPush(topic)) {
    Serial.println("[REQ] Solicitud encolada.");
  } else {
    Serial.println("[ERR] FIFO llena: solicitud descartada.");
  }
  fifoPrint();
  return;
}

// 2) Cancelacion (robot perdio el marcador antes del permiso)
if (msg.indexOf("Cruce") != -1 && msg.indexOf("Cancelado") != -1) {
  // Si era el permiso pendiente, cancelarlo tambien
  if (permisoPendiente && strcmp(topicPendiente, topic) == 0) {
    permisoPendiente = false;
    Serial.print("[NO] Permiso pendiente cancelado para: ");
    Serial.println(topic);
  } else {
    if (fifoRemove(topic)) {
      Serial.print("[DEL] Eliminado de FIFO: ");
      Serial.println(topic);
    }
  }
  fifoPrint();
  return;
}

// 3) Cruce liberado (robot supero el 2 marcador)
if (msg.indexOf("Cruce") != -1 && msg.indexOf("Liberado") != -1) {
  if (cruceOcupado && strcmp(topic, robotEnCruce) == 0) {
    cruceOcupado = false;
    robotEnCruce[0] = '\0';
    Serial.println("[GO] Cruce liberado");
  } else {
    Serial.println("[WARN] Liberacion ignorada: robot no coincide");
  }
  return;
}
}
```

```
/* ----- SETUP ----- */
void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("=== CONTROLADOR DE CRUCE - version sin tiempos ===");

  WiFi.mode(WIFI_AP);
  WiFi.softAP(AP_SSID, AP_PASS);

  Serial.print("[IP] IP del broker: ");
  Serial.println(WiFi.softAPIP());

  mqtt.begin();
  mqtt.subscribe(TOPIC_R1, manejarMensaje);
  mqtt.subscribe(TOPIC_R2, manejarMensaje);

  Serial.println("[OK] Broker MQTT listo");
}

/* ----- LOOP ----- */
void loop() {
  mqtt.loop();

  // Sacar siguiente de la FIFO cuando el cruce este libre
  if (!cruceOcupado && !permisoPendiente) {
    if (fifoPop(topicPendiente, sizeof(topicPendiente))) {
      permisoPendiente = true;
      tSolicitud = millis();
      Serial.print("[WAIT] Siguiendo en FIFO: ");
      Serial.println(topicPendiente);
      fifoPrint();
    }
  }

  // Dar permiso tras el pequeno retardo de estabilizacion
  if (permisoPendiente && (millis() - tSolicitud >= RETARDO_OK_MS)) {
    permisoPendiente = false;
    concederPermiso(topicPendiente);
  }
}
```